

Reverse Engineering the RVQ Audio Encoder of MiniMax Music 3 via Reverse Distillation

Munaza Ashraf¹

Bghira²

¹github.com/MunazaAshraf10 ²github.com/bghira/SimpleTuner

Abstract

MiniMax Music 3 plans a track as eight residual vector quantised (RVQ) code streams at 25 Hz and renders them with a diffusion transformer over the latents of a continuous audio autoencoder. The encoder that maps audio back to those codes was never released, so a reference recording cannot condition the model. We learn a functional surrogate of that missing map by reverse distillation: we generate 2,972 tracks with the model itself, keep the sampled codes together with the generator’s top-50 code distributions, and train an encoder from the frozen autoencoder latents to the codes. Two design choices account for most of the result. First, the generator’s code timeline is not a fixed ratio of the latent timeline but is stitched from 200-frame rollout chunks, and we recover the exact frame-to-latent correspondence and pool latents with it. Second, the acoustic codebooks are conditionally dependent across depth; replacing seven independent readouts by a decoder that is causal across codebook depth raises the downstream condition-replay cosine on 130 held-out generated tracks from 0.7698 to 0.8748 (paired bootstrap interval of the gain [0.101, 0.109]; community baseline 0.6633, true codes 0.9999) while exact-token agreement barely moves, suggesting substantial functional redundancy in the code space.

We release four encoders of 41M to 169M parameters, trained under maximal update parametrisation, together with the corpus. The recommended encoder reproduces its published token metrics to within 0.001 when re-evaluated on the same held-out split and encodes six minutes of stereo audio in 5.6 s on one RTX 3090. All evaluation is on model-generated audio and stops before the diffusion stage; calibration of the metric against random codes and a study on recorded music are the open validations.

1. Introduction

Codec language models generate audio as sequences of discrete codes from a neural audio codec and render those codes with a decoder [6, 8, 39]. MiniMax Music 3 [28] follows this design with a twist: a Qwen3 language model [31] plans one semantic codebook of 16,384 entries and an RVQ depth decoder fills in seven acoustic codebooks of 1,024 entries, eight integers per 40 ms frame, which a condition encoder turns into the conditioning of a flow-matching diffusion transformer [24, 30] over 128-channel latents of a Descript-style audio autoencoder [22], the DAV. The released weights cover text-to-music generation only. The component that would let

audio enter the code space, an audio-to-RVQ encoder, is not released, and the released inference code contains no path that uses one.

This paper learns a functional surrogate of that component and shows, by replaying its codes through the official model, that it recovers most of the conditioning signal on the generator’s own audio.¹ Our premise is that the generator provides natural supervision for its inverse: every track it produces comes with the codes that produced it and with the generator’s distribution over codes at every frame, from the language model for the semantic codebook and from the depth decoder for the acoustic ones. Encoding the waveform of such a track through the frozen DAV encoder and regressing the codes from the latents is a supervised problem with exact labels. We use the term reverse distillation descriptively for this setup: the generator supplies paired outputs and their discrete causes, and its predictive distributions provide soft targets for learning an approximate inverse mapping. It is a benign instance of the model-extraction setting in which a withheld component is recovered from query access to the released system [7, 35].

Three parts of the solution are of independent interest.

Timeline contract. The code stream runs at 25 Hz and the latents at 86.1328125 Hz, so the nominal stride is $441/128 = 3.4453125$ latents per frame. The generator, however, renders in 200-frame chunks with a 100-frame hop and stitches the latents with an integer hop of 345, and each chunk owns its frames from an offset of 25. A fixed stride therefore drifts against the true timeline by a full frame within a few chunks. We derive the exact boundary sequence, confirm it against the generator’s own stitching tables, and pool latents into frames with the resulting variable spans (Sec. 4.1).

Causal decoding across codebook depth. Residual codebooks are conditionally dependent: the meaning of codebook k depends on which codes were chosen below it. Independent readouts show a monotone collapse of accuracy with depth. A small decoder that is autoregressive across depth, conditioned on the frame context and the codes below, removes most of the collapse under teacher forcing and lifts the downstream metric by 0.10 (Sec. 5.2).

A downstream functional metric. Exact agreement with the sampled code tuple is a weak proxy. Replay pre-

¹The code and the weights and corpus are public.

dicted codes through the official language-model path and comparing the resulting condition embedding with the stored one (condition-replay cosine) separates encoders that exact-token metrics cannot: the depth decoder loses 0.32 points of free-running acoustic top-1 and gains 0.105 of replay cosine against the structurally matched encoder (Sec. 7).

Our contributions are:

- a 2,972-track reverse-distillation corpus with sampled codes, top-50 teacher distributions and chunk stitching tables;
- the exact frame-to-latent timeline of MiniMax Music 3 and the pooling operator it implies;
- four encoders, from a 41M-parameter baseline to a 169M-parameter model with a causal depth decoder, trained under maximal update parametrisation [43] with a hard cross-entropy plus top-50 distillation objective [17], reaching 0.8748 condition-replay cosine on held-out generated tracks against a 0.6633 community baseline, with paired bootstrap intervals on every difference;
- evidence that downstream conditioning is substantially more tolerant to alternative code tuples than exact-token agreement suggests.

2. Background and Related Work

2.1. Neural audio codecs

Vector-quantised autoencoders [37] turned audio into discrete tokens, and neural codecs made the tokens efficient: SoundStream [45], EnCodec [11], HiFi-Codec [42] and the Descript Audio Codec [22] compress waveforms with convolutional stacks descended from WaveNet [36], trained with reconstruction and adversarial losses. MiniMax Music 3’s DAV follows the Descript encoder layout with the Snake activation [47], but it is a continuous variational autoencoder: its 128-channel posterior mean is the input of the diffusion transformer, and the eight discrete code streams are produced by a separate tokenizer that was not released.

2.2. Residual vector quantisation

Residual vector quantisation (RVQ) encodes a vector as a sum of codebook entries chosen in sequence, each codebook quantising the residual left by the previous ones [45]. The first codebooks carry coarse content and the later ones refine it, and the meaning of a code at depth k depends on the codes chosen below it. Codec language models exploit this hierarchy with coarse-to-fine stages [6], delayed patterns [8] or a depth transformer [12]. Our encoder does not learn a quantiser; it is a classifier over frozen codebooks whose entries we never observe, and Sec. 8 shows that the conditional dependence across depth survives that framing.

2.3. Audio tokenization for music generation

Jukebox [13] modelled music with transformers over VQ-VAE codes. AudioLM [6] and MusicLM [1] separated semantic tokens, distilled from self-supervised speech representations such as HuBERT [19], from acoustic codec tokens. SpeechTokenizer [46] and Mimi [12] fold the semantic content into the first RVQ layer of the codec itself. MiniMax Music 3 uses one semantic codebook of 16,384 entries and seven acoustic codebooks of 1,024 entries, the same split, and renders the codes with latent diffusion [18, 24] over a transformer backbone [14, 30], as recent long-form music generators do [15].

2.4. Existing music generation systems and model extraction

Open music generators release their tokenizer with their language model [1, 8]; MiniMax Music 3 [28] releases the generator but not the audio-to-code path. Recovering a model or one of its components from query access has been studied as model stealing: prediction APIs leak decision boundaries [35], black-box functionality can be copied into a knock-off network [29], and hidden dimensions of a production language model can be read out of its logits [7]. Our problem is the benign counterpart: the released system exposes its outputs and its internal code distributions, and we recover the one component the release withholds by training on those outputs, with the temperature-scaled distillation of Hinton et al. [17] on the teacher’s own samples [4, 20]. All encoders are trained under maximal update parametrisation [43], and the gap we measure between teacher-forced and greedy decoding is the exposure bias of sequence models [3, 33] across codebook depth rather than time.

3. MiniMax Music 3 Audio Representation

3.1. System interface and observations

We describe the pipeline as it can be observed from the released weights and inference code. A Qwen3-based language model [31] consumes a caption and lyrics and emits, per 40 ms frame, a semantic code $c_0 \in \{0, \dots, 16383\}$ or an end-of-track symbol (id 16,384). An RVQ depth decoder, a small transformer with a key-value cache over depth, emits seven acoustic codes $c_1, \dots, c_7 \in \{0, \dots, 1023\}$ conditioned on the language-model hidden state and the codes below. The language model runs in windows of 200 frames with a hop of 100 frames, so every frame after the first 100 is generated twice, and the later chunk owns a frame from its local index 25 onward. A condition encoder maps the code stream to a conditioning sequence; a flow-matching diffusion transformer renders 128-channel DAV latents at 86.1328125 Hz from that conditioning, chunk by chunk, and the chunks are stitched with a hop of 345 latents; the DAV decoder returns 44.1 kHz stereo audio.

The release contains the language model, the depth decoder, the condition encoder, the diffusion transformer, and the DAV

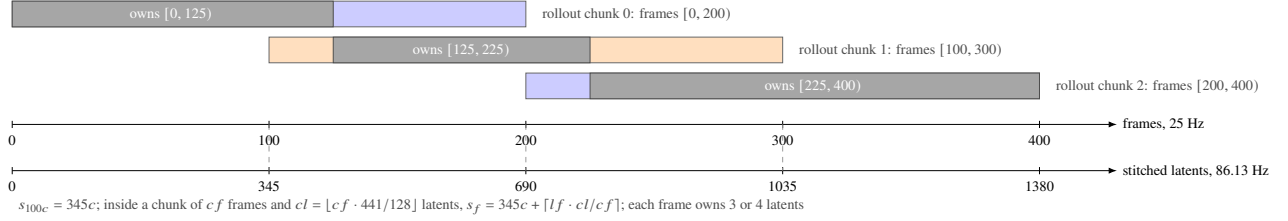


Figure 1. The stitched timeline. Rollout chunks of 200 frames at a 100-frame hop; each chunk after the first owns its frames from local index 25; latents are stitched with a hop of 345, so a frame owns three or four latents and the boundaries follow Eq. 1.

encoder and decoder. It contains neither an audio-to-code path nor the tokenizer that defines the codebooks. A reference recording therefore cannot be expressed in the code space the language model consumes.

3.2. Initial characterisation

Three properties of the interface shape everything that follows. The generator exposes, for every track it produces, the sampled codes and a top-50 distribution at every frame and codebook, from the language model for the semantic codebook and from the RVQ depth decoder for the seven acoustic ones, so the inverse map from audio to codes has exact labels and soft targets for free. The code stream at 25 Hz and the latent stream at 86.1328125 Hz stand in a non-integer ratio of $441/128 = 3.4453125$, and the chunked rollout with its integer stitching hop of 345 makes the true correspondence piecewise rather than a fixed stride. And the acoustic codebooks are produced by a decoder that conditions each on the codes below, so the codes of one frame are not independent labels.

3.3. Research questions

We ask (i) what the exact correspondence between latent and code timelines is and whether it can be reconstructed without the generator’s own stitching tables; (ii) whether an encoder trained only on generated tracks can recover codes that the generator accepts as equivalent to its own; (iii) how much of the acoustic code stream can be predicted at all, given that the same audio admits many compatible code tuples; and (iv) which architectural choice, width, auxiliary alignment, or conditioning across codebook depth, moves the downstream behaviour.

4. Reverse-Engineering Methodology

4.1. Temporal resolution analysis

Let n be the number of frames of a track. Frame t covers a span $[s_t, s_{t+1})$ of stitched latents, and the $n + 1$ boundaries $s_0 \leq \dots \leq s_n$ are the entire contract between the two timelines (Fig. 1).

Exact rule. From the generator’s stitching table, chunk 0 owns frames $[f_0, f_1 + 25)$, chunk $i > 0$ owns $[f_i + 25, f_{i+1} + 25)$, and the last chunk owns through its own end. A chunk that

owns *owned* frames starting at frame g_0 and that kept *kept* latents $[a, b)$ maps its frame g to $s_g = a + [(g - g_0) \text{kept} / \text{owned}]$, and a later chunk rewrites the boundary it shares with the previous one.

Nominal rule. A reference recording carries no stitching table, so inference must reconstruct the timeline from the frame count alone. With $W = \max(1, \lfloor (n - 1) / 100 \rfloor)$ chunks, frame f belongs to chunk $c = \text{clamp}(\lfloor (f - 25) / 100 \rfloor, 0, W - 1)$; a chunk of $cf = \min(200, n - 100c)$ frames yields $cl = \lfloor cf \cdot 441 / 128 \rfloor$ latents, and with $lf = f - 100c$,

$$s_f = 345c + \left\lceil \frac{lf \cdot cl}{cf} \right\rceil. \quad (1)$$

The rule gives $s_{100} = 345$, $s_{200} = 690$, $s_{4500} = 15,524$, and every span holds three or four latents; it agrees with the stitching tables on every full chunk and interpolates the shorter last chunk. A fixed stride of 3.4453125 places frame 4,500 at latent 15,504, twenty latents and almost six frames early. The 345-latent hop, the priming row and the last-chunk rule were also identified independently by a community encoder [34], so the contract is a property of the generator rather than of our reading of it.

4.2. Latent representation analysis

The DAV encoder maps 44.1 kHz audio to 1,024 channels at hop 512 and a posterior mean of 64 channels per stereo side, 128 channels at 86.1328125 Hz, left channels first. We use these posterior means, not a mel spectrogram, as the encoder input: they are the representation the diffusion transformer already consumes, and re-encoding the generated waveform through the frozen DAV encoder reproduces the generator’s stored latents with 0.986 mean cosine at zero lag, so the input side of the inverse map is essentially exact. The stored flow latents themselves are not used for training because a real recording would not have them.

4.3. Quantiser structure identification

The observable code stream has one semantic codebook of 16,384 entries plus an end-of-track id 16,384 that never appears in audio, and seven acoustic codebooks of 1,024 entries. Row 0 of every stored code tensor is a priming row that was never emitted, so semantic frame i is supervised by row $i + 1$.

Table 1. The four encoders. Parameter counts are those of the released weights.

Model	Change from the previous model	Width	Parameters
v1	baseline, 7 independent acoustic readouts	512	40,978,944
v2	shared encoder widened, 8 to 17 heads	1,088	154,736,064
v3	v2 plus training-only MERT alignment	1,088	154,736,064
v4	v2 encoder plus causal depth decoder	1,088	169,008,576

The depth decoder that produces the acoustic codes is a two-layer causal transformer over the depth axis with a key-value cache, which fixes the direction of dependence: codebook k is drawn conditioned on the language-model state and codebooks 0 to $k - 1$, never on later ones.

4.4. Codebook analysis

Because the codebook entries are never observed, the codebooks can only be analysed through their behaviour. Two facts emerge from training independent readouts (Sec. 8): top-1 agreement declines monotonically with depth, from 11 to 12% at codebook 1 to under 3% at codebook 7, and the same audio supports many code tuples that condition the generator identically, so exact agreement with the one sampled tuple is bounded well below the agreement that would matter downstream. Both facts point to a decoder that is conditioned across depth rather than seven independent classifiers.

4.5. Architecture inference

From the above we infer the shape of the missing encoder: a front end at the latent rate with a receptive field of a few frames, a pooling step that follows the stitched timeline exactly, a bidirectional context model over a window, a semantic readout, and a decoder over depth that mirrors the generator’s own. Section 5 instantiates that inference.

5. Reconstructed Audio Encoder

We train four encoders that share one design and differ in one change each (Tab. 1). v1 is the baseline; v2 widens the shared encoder; v3 adds a training-only auxiliary objective to v2; v4 keeps the v2 encoder and replaces the independent acoustic readouts by a decoder that is causal across codebook depth. Every model is trained from scratch; no weights are transferred between versions.

5.1. Architecture

The input is the DAV posterior mean $h \in \mathbb{R}^{L \times 128}$ of a window (Fig. 2). A convolutional stem (kernel 7, 128 to d channels) and three residual blocks $x + \text{conv}_1(\text{gelu}(\text{conv}_3(\text{gelu}(\text{GN}_1(x))))$ with dilations 1, 3 and 9 [44] operate at the latent rate, with GELU activations [16]. GroupNorm with one group [40] normalises channels and time jointly, so zero-padded latents behave identically in batched and single-window evaluation. Frame features are the

mean of the latent features a frame was rendered from,

$$P_{t,l} = \frac{1}{s_{t+1} - s_t} \text{ for } l \in [s_t, s_{t+1}), \quad 0 \text{ otherwise,} \quad (2)$$

so every row of P sums to one; P maps the result to $T \leq 128$ frames, learned positions are added, and eight bidirectional pre-norm transformer layers [38, 41] (width d , head dimension 64, feed-forward $4d$, dropout 0.1) followed by a Layer-Norm [2] give frame features $y \in \mathbb{R}^{T \times d}$. The window is 128 frames, 5.12 s, with no state across windows.

5.2. Quantisation pipeline

The semantic readout maps y to 16,384 logits. v1 to v3 read the seven acoustic codebooks from y with independent linear heads. v4 replaces them with a decoder that is autoregressive across codebook depth and runs once per frame: the sequence $[W_{\text{ctx}}y_t, e_0(c_0), e_1(c_1), \dots, e_6(c_6)]$ plus learned depth positions passes through two causal pre-norm layers of width 512 (8 heads, feed-forward 2,048), and head j reads position $j + 1$ to predict codebook $j + 1$. Codebook k therefore sees the frame context, the semantic code and the codebooks below k , and nothing above. Training teacher-forces the sampled codes in one pass; inference feeds back greedy predictions in seven sequential passes, and the argmax of each head is the emitted code. The decoder adds 22.1M parameters to the 147M of the shared encoder and semantic readout.

Let V_k be the size of codebook k . With $K = 8$ codebooks, sampled codes $c_{t,k}$, and the stored top-50 ids i and teacher logits z_T , the objective is

$$\mathcal{L} = \frac{1}{K} \sum_k \text{CE}_k + w_{\text{KL}} \frac{1}{K} \sum_k \text{KL}_k, \quad (3)$$

$$\text{KL}_k = \tau^2 \sum_{i \in \text{valid}} p_T(i) (\log p_T(i) - \log p_S(i)),$$

where $p_T = \text{softmax}(z_T/\tau)$ over the valid stored ids, p_S is the student softmax over the whole vocabulary at temperature τ gathered at those ids, and ids outside $[0, V_k)$, notably the end-of-track id and padding, are dropped before renormalising the teacher [17]. All models use $w_{\text{KL}} = 0.25$ and $\tau = 1$. For v4 the acoustic terms are teacher-forced, so its loss is conditional and not comparable with v1 to v3.

5.3. Implementation

Because each row of P is one contiguous constant block, Ph is a segment mean and is computed in $O(Ld)$ by scattering each latent into its owning frame and dividing by the span, rather than in $O(TLd)$ as a dense product. Fused scaled dot-product attention [9, 10, 32] is used in the temporal stack and an unfused kernel in the depth decoder, a choice decided by measurement (Sec. 7). All models use maximal update parametrisation [43] with width multiplier $m = d/128$: readouts compute $W(x/m) + b$, attention logits are scaled by $8/d_h$, and matrix-like weights take learning rate lr/m and weight decay $\text{wd} \cdot m$ under AdamW [21, 26]; biases with a width-sized fan-in and non-zero readouts are scaled by $\sqrt{m}$ at initialisation,

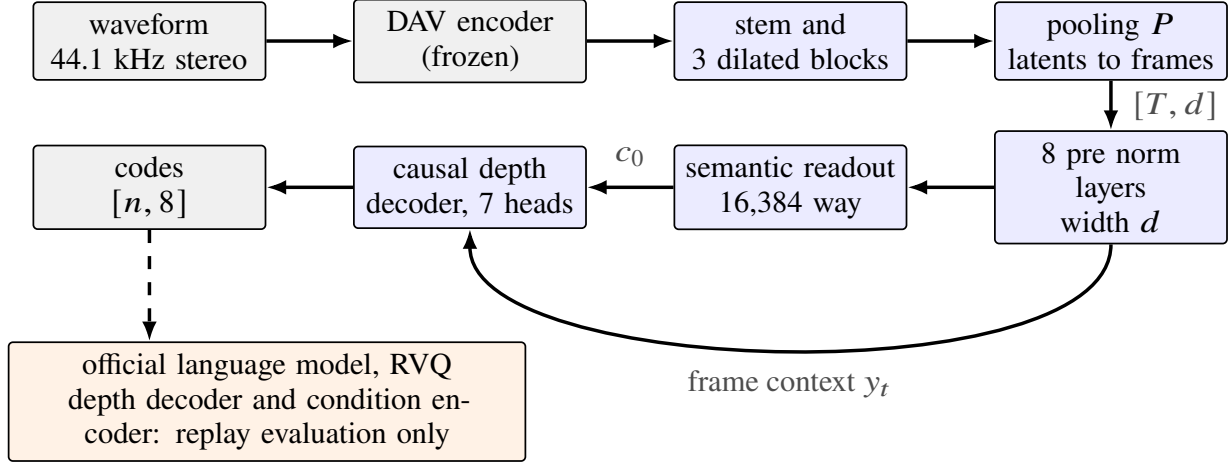


Figure 2. Waveform to codes. The DAV encoder yields $[L, 128]$ latents, pooling yields $[T, d]$ frame features. Grey: frozen and released components. Blue: the encoder trained here. Orange: the official path used only to compute condition-replay cosine.

and the semantic readout starts at zero. The DAV encoder is evaluated in 30 s segments with 1 s margins, so peak memory is flat for any track length.

6. Experimental Setup

6.1. Dataset

We issued 4,000 generation requests to the released pipeline (30 flow steps, seed 20260815, 180 s requested per track, 3,772 with lyrics and 228 instrumental); 2,972 of them completed (2,775 with lyrics, 197 instrumental) and form the corpus [5]: 91.84 h of audio, mean length 1 min 51 s, with 79.6% of tracks ending naturally on the end-of-track code. A fifth of the requests draw a deterministic sampling variant (temperature 0.85 or 1.15 with flow guidance 1.5 or 2.0); the rest use the defaults (temperature 1.0, top-50 sampling, guidance 1.5, flow guidance 1.7). Each track is stored with its audio, the sampled codes, the generator’s top-50 ids and logits per frame and codebook (language model for the semantic codebook, depth decoder for the acoustic ones), the condition embeddings, the flow latents, and the chunk stitching table. A deterministic hash of the track assigns 5% of tracks to a held-out split, so every window of a track lies in one split: 2,837 training records and 135 held-out records. The five held-out records without an exact stitching table (shards 12, 53, 73, 86 and 134) belong to the first generation batch, produced before per-chunk spans were recorded; their exclusion is determined by batch, not by content or by any failure of the encoder, and the same rule removes 178 training records. The training input is the waveform re-encoded through the frozen DAV encoder.

Table 2 describes the content. Prompts are short genre captions, four of which (rock, pop, blues, jazz) cover 88% of the corpus, so the held-out prompts are almost all also training prompts; lyrics are drawn from a tag-conditioned song source, and 22 of the 135 held-out tracks share their lyrics verbatim with a training track while differing in prompt, seed and sampling. The corpus is therefore narrow in caption space and

Table 2. Content statistics of the reverse-distillation corpus.

Statistic	Value
tracks (train / held-out)	2,972 (2,837 / 135)
with exact stitching table (train / held-out)	2,659 / 130
lyric-bearing / instrumental	2,775 / 197
distinct prompts	307
four most frequent prompts	rock (705), pop (658), blues (622), jazz (620)
prompt length in words, median (max)	1 (187)
lyric length in lines, median (mean)	33 (35.5)
held-out prompts also used in train	124 of 135
held-out lyrics identical to a training track	22 of 135

the held-out split tests generalisation across generations, not across musical styles.

6.2. Evaluation metrics

Evaluation uses the 130 exact-alignment held-out tracks, 2,768 deterministic 128-frame windows. *Token agreement*: top-1 and top-5 inclusion of the sampled code, per codebook and pooled over the seven acoustic books; models with a depth decoder are scored free-running (greedy feedback, the inference condition) and teacher-forced. *Condition-replay cosine*: the encoder’s free-running codes are teacher-forced through the official language model and depth decoder, the hidden states pass through the official condition encoder with the recorded stitching, and the result is compared with the stored condition embedding, mean cosine over frames, then over tracks. True sampled codes replayed the same way score 0.9999 and are the control. The replay metric stops before the diffusion transformer; it is not a waveform or listening score, and its floor is not calibrated here: no score for random, shuffled or partially correct code tuples was available through the official path (Sec. 10). Every architecture was trained once, with seed 42, so differences of the order of the bootstrap intervals in Tab. 5 are not interpreted.

6.3. Baselines

The external baseline is the community encoder of Serveurperso [34], a 41M model trained independently

Table 3. Condition-replay cosine on the 130 exact-alignment held-out tracks.

Model	Parameters	Mean	Std	5th pct	95th pct
Serveurperso v1 (community)	40,978,944	0.6633	0.0222	0.6281	0.6963
v1, 41M	40,978,944	0.7624	0.0195	0.7345	0.7904
v2, 155M	154,736,064	0.7698	0.0191	0.7430	0.7986
v3, 155M + MERT	154,736,064	0.7703	0.0193	0.7416	0.8005
v4, 169M + depth	169,008,576	0.8748	0.0157	0.8453	0.8938
true codes (control)		0.9999	0.0002	0.9998	1.0000

on a separate 550-track corpus with its own alignment and evaluation stack, scored under our protocol. Within our family, v1 is the size-matched baseline for that encoder and each of v2, v3 and v4 is the baseline for the next.

6.4. Implementation details

All runs use 4 L40S GPUs, batch 16 per rank, 20 epochs (17,660 optimizer steps), learning rate 3×10^{-4} , weight decay 0.01, gradient norm clipped at 1.0, bfloat16 autocast [27], random 128-frame crops and seed 42. v1 used a cosine schedule of half period 500 that reheats fully every 1,000 steps [25]; v2 to v4 used 500 steps of linear warm-up followed by linear decay to 10^{-7} . v3 added a training-only cosine alignment term between the layer-4 hidden state and layer 9 of a frozen MERT [23] model, weighted 0.5 until 70% of training and annealed to zero by 90%; its projection is not exported, so v2 and v3 share an architecture. The supplementary material (Sec. 14) lists every hyperparameter.

7. Results

7.1. Reconstruction quality

The quantity an encoder must reconstruct is the conditioning the generator would have received from the true codes, and Tab. 3 reports it as condition-replay cosine for every model on the 130 held-out tracks. The first column names the model and the second its parameter count; the mean is the average over tracks of the per-track cosine, the standard deviation is taken across tracks, and the 5th and 95th percentile columns bound the central ninety percent of tracks; the full range of each model is drawn in Fig. 3. The last row is the control obtained by replaying the true sampled codes through the same path, which scores 0.9999 and marks the ceiling of the metric. Three steps are visible down the rows. The 41M baseline trained here exceeds the independent community encoder of the same size by 0.099 in mean cosine, which we attribute to the exact timeline and to the distillation term. Widening the shared encoder to 155M adds 0.007 and MERT alignment adds 0.0004, both within one standard deviation of the run before them. The causal depth decoder adds 0.105 over v3, and the whole distribution moves: its 5th-percentile track (0.845) exceeds the best track of any independent-head model (0.836), and the standard deviation shrinks from 0.019 to 0.016.

On the input side of the map, re-encoding the generated waveform through the frozen DAV encoder reproduces the generator’s stored latents with 0.986 mean cosine, so the re-

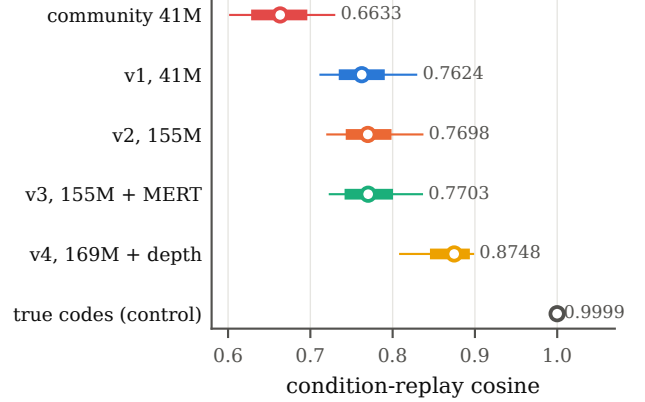


Figure 3. Condition-replay cosine over the 130 held-out tracks: mean (marker), 5th to 95th percentile (bar), min to max (line). The v4 (depth-causal) distribution is separated from every independent-head model.

maining gap between any model and the control is in the codes, not in the latents that feed the encoder.

Figure 3 draws the distributions of Tab. 3 as one row per model. The marker is the mean, the thick bar spans the 5th to 95th percentile, and the thin line spans the full range from the worst to the best track. Read from top to bottom, the rows step upward from the community encoder through v1, v2 and v3, whose bars overlap almost entirely, to v4, whose bar does not overlap any other model’s bar; the control row sits alone at the right edge. The figure makes the two facts of the table visible at once: width and alignment move the mean by less than the width of a bar, and the depth decoder moves the entire population of tracks, not only the average.

7.2. Token agreement

Table 4 scores the four models on exact agreement with the sampled codes at the matched step 17,500, the last checkpoint common to all runs. The loss column is the training objective at KL weight 0.25. The four accuracy columns give top-1 and top-5 inclusion of the sampled code, first for the semantic codebook and then pooled over the seven acoustic codebooks. Semantic top-1 rises from 41.0% to 43.2% and semantic top-5 from 78.4% to 80.5% across the family, while pooled acoustic top-1 stays between 7.2% and 7.7% for every model; the acoustic columns are the ones that do not follow the replay metric of Tab. 3, which is the observation Sec. 8 builds on.

The v4 loss is lower than the other three rows because the acoustic terms of v4 are conditioned on the true earlier

Table 4. Exact-token metrics at the matched step 17,500 (percent).

Model	Loss	Sem. top-1	Sem. top-5	Ac. top-1	Ac. top-5
v1, 41M	5.3379	41.03	78.38	7.17	20.94
v2, 155M	5.2646	42.86	80.17	7.62	21.98
v3, 155M + MERT	5.2602	43.03	80.48	7.65	22.02
v4, 169M + depth	3.9882	43.16	80.51	7.30	19.88

Table 5. Paired differences in condition-replay cosine over the same 130 tracks, with percentile bootstrap intervals (10,000 resamples).

Comparison	Mean difference	95% interval	Tracks improved
v1 minus community 41M	+0.0991	[+0.0968, +0.1014]	130 of 130
v2 minus v1	+0.0074	[+0.0070, +0.0078]	130 of 130
v3 minus v2	+0.0004	[+0.0000, +0.0008]	77 of 130
v4 minus v2	+0.1050	[+0.1014, +0.1086]	130 of 130
v4 minus v3	+0.1045	[+0.1009, +0.1082]	130 of 130

codes during scoring, so they are not comparable with the independent-head rows and should not be read as a better fit. Re-evaluating the released weights on the same held-out split reproduces every published row within 0.003 in loss and 0.001 in accuracy (Sec. 13).

7.3. Compression characteristics

The code stream carries $14 + 7 \times 10 = 84$ bits per frame at 25 Hz, 2.1 kbit/s, against 128 latent channels at 86.13 Hz on the DAV side and 1,411 kbit/s for the 16-bit stereo waveform. The encoder is therefore a lossy map by construction, and the tolerance of the downstream conditioning to alternative tuples (Sec. 8) is what lets a 2.1 kbit/s stream condition the generator at 0.87 cosine. Encoding runs at $64\times$ real time on one RTX 3090, 5.6 s for six minutes of stereo audio, four fifths of it in the frozen DAV encoder.

7.4. Ablation studies

Each model in Tab. 1 differs from the one before it by one named change, so the rows of Tab. 3 read as an ablation, and because every model is scored on the same 130 tracks the differences are paired. Table 5 gives each difference of interest with a percentile bootstrap interval: the first column names the pair, the second is the mean of the per-track differences, the third is the 95% interval from 10,000 resamples of the 130 tracks, and the last column counts the tracks on which the first model beats the second. Three of the five comparisons improve every one of the 130 tracks: v1 over the community encoder (+0.099), v2 over v1 (+0.007) and v4 over either v2 or v3 (+0.105 and +0.1045). The MERT comparison is the exception, +0.0004 with an interval that touches zero and only 77 of 130 tracks improved, which is why we describe it as negligible for a single run rather than as an improvement.

Two caveats limit what the ablation isolates. The larger v2 configuration improves every token metric over v1 by one to two points and replay cosine by 0.007, but v1 also used a different learning-rate schedule (a reheating cosine against linear warm-up and decay) and a different head count, so this comparison does not isolate width, and we read it only as evidence that capacity was not the main limit. MERT alignment (v2 to v3) changes token metrics by 0.03 to 0.3 points and replay by 0.0004, too little to justify a training dependency, and v4 therefore builds on the v2 encoder rather than on v3. The causal depth decoder (v4 against v2, the structurally matched comparison) improves semantic agreement marginally, *lowers* free-running acoustic top-1 by 0.32 points, and raises replay cosine by 0.105, the only change that moves

the downstream metric substantially. v4 also carries 22.1M more parameters than v2 in the decoder; a parameter-matched independent-head model or a non-causal decoder of the same size, which would separate conditioning from capacity, was not trained (Sec. 10).

Two implementation choices were also ablated by measurement. Fused attention is $3.6\times$ faster forward and $2.8\times$ faster backward at the temporal stack’s shape (batch 16, 17 heads, length 128) and $2.9\times$ and $3.8\times$ slower at the depth decoder’s shape (batch 2,048, 8 heads, length 8), so the two stacks use different kernels. The segment form of the pooling operator is $11\times$ to $18\times$ cheaper to assemble per batch than the dense matrix and equal on device (Sec. 13).

8. Analysis

8.1. Representation structure

The decisive observation is the split between exact-token and downstream metrics. v4 loses 0.32 points of free-running acoustic top-1 against v2 and gains 0.105 of replay cosine; its 5th-percentile track beats the best v2 or v3 track. This suggests substantial functional redundancy in the code space: many code tuples appear to map to similar condition embeddings, and an encoder can be functionally close while disagreeing with the one tuple the generator happened to sample. Exact agreement is bounded by that sampling, not by the encoder, which is consistent with the semantic head saturating near 43% top-1 and 80% top-5 across every model while replay keeps improving. We have not quantified equivalence classes of tuples directly; the claim rests on the divergence of the two metrics.

Table 6 makes the same point from inside the v4 model by scoring it under the two conditions it can be run in. Free-running, the first numeric column, is the inference condition: each acoustic head sees the greedy predictions of the heads below it. Teacher-forced, the second column, is the training condition: each head sees the sampled codes below it. The two semantic rows are identical, because the semantic readout does not depend on the acoustic codes. The two acoustic rows differ by a factor of 2.5 at top-1 (7.3% against 18.4%) and 2.2 at top-5 (19.9% against 44.6%): the decoder has learned the conditional structure of the codebooks, and feeding back its own earlier choices is what costs exact agreement. Since the replay cosine of Tab. 3 is measured on the free-running codes, the 0.8748 is achieved with the 7.3% column, not the 18.4% one.

8.2. RVQ codebook behaviour

Because the codebook entries are never observed, the codebooks can only be characterised by how predictable each one is from audio. Table 7 gives top-1 agreement for every codebook separately at the matched step 17,500. The first column is the codebook index, with 0 the semantic stream and 1 to 7 the acoustic residuals in order of depth; the next three columns are the independent-head models v1, v2 and v3; the last two

Table 6. v4 at the final checkpoint, free-running against teacher-forced (percent).

Metric	Free-running	Teacher-forced
semantic top-1	43.17	43.17
semantic top-5	80.51	80.51
acoustic top-1	7.29	18.42
acoustic top-5	19.88	44.55

are v4 free-running and v4 teacher-forced. Reading down any of the first three model columns shows the depth gradient: from 11 to 12% at codebook 1 to under 3% at codebook 7, and the three columns differ by at most 0.7 points at any depth. Reading down the teacher-forced column shows what the decoder learned, 27.0% at codebook 1 and still 11.9% at codebook 7, more than four times the independent heads at depth. The free-running column sits above v3 at codebooks 1 and 2 and falls below it from codebook 4 on, where an early disagreement has propagated through the conditioning.

The monotone decline of the independent heads is consistent with the conditional dependence induced by residual quantisation, where deeper codebooks refine a representation determined by the codes below and an independent head cannot see which were chosen. Under greedy feedback most of the teacher-forced gain is lost in exact agreement, the exposure bias of sequence models [3, 33] across depth rather than time. The replay metric suggests that many of the alternative tuples remain functionally compatible with the generator: although they differ from the sampled tuple, they induce substantially more similar conditioning than exact-token agreement implies.

8.3. Musical information across levels

Figure 4 plots the seven acoustic columns of Tab. 7 against codebook index. The three solid curves of the independent-head models lie on top of one another, which is the visual form of the width and MERT ablations having no effect on the depth gradient. The dashed curve is the teacher-forced v4 decoder, the conditional structure it learned; the solid v4 curve is what survives greedy decoding, and the vertical distance between the two at each codebook is the exposure gap, widening from 12 points at codebook 1 to 10 points at codebook 7 while both curves fall. The figure also shows where the free-running v4 curve crosses below the independent heads, between code-

Table 7. Top-1 agreement by codebook at step 17,500 (percent).

Codebook	v1	v2	v3	v4 free-running	v4 teacher-forced
0	41.03	42.86	43.03	43.16	43.16
1	11.27	11.96	12.04	14.72	27.04
2	11.92	12.56	12.62	13.02	24.49
3	9.34	10.00	9.98	9.46	19.32
4	7.54	8.07	8.17	6.56	16.66
5	4.45	4.71	4.74	3.41	15.46
6	3.08	3.29	3.26	2.16	14.10
7	2.58	2.77	2.77	1.76	11.88

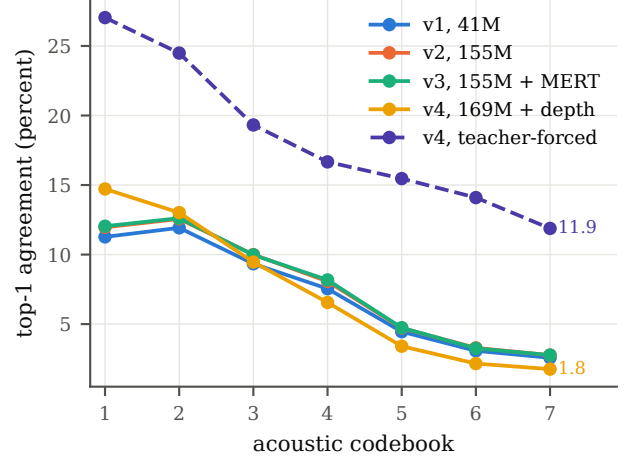


Figure 4. Top-1 agreement by acoustic codebook at step 17,500. Independent heads (v1 to v3) collapse with depth; the depth-causal decoder (v4) does not when teacher-forced and partly does under greedy feedback.

books 3 and 4.

The semantic codebook is the most predictable stream from audio (43% top-1, 80% top-5 over 16,384 entries) and the one the language model plans directly; it carries the content that a text prompt determines. The acoustic codebooks refine timbre and detail and are progressively less determined by the audio alone: given the codes below, codebook 1 is predicted at 27% and codebook 7 at 12%, and without them the later books approach chance. Width and MERT alignment, both of which enrich the frame representation, barely move any level; conditioning across levels moves all of them. The information that matters appears to lie in the joint structure of the tuple rather than in any level in isolation.

8.4. Implications for generative modelling

For reference-conditioned generation the practical recipe follows from the analysis: constrain the semantic stream, which the encoder predicts well and which the language model consumes directly, and let the generator draw the acoustic codes itself. The released integration restricts every fifth generated semantic code to the encoder’s top-5 candidates and generates all seven acoustic codebooks with the official depth decoder. More generally, an encoder for a hierarchical code space should be scored by what the downstream model does with its codes, and should decode across depth rather than read levels independently.

9. Discussion

Reverse distillation works because the generator supplies exact labels and soft targets for its own outputs, and because two structural facts about the interface, the stitched timeline and the dependence across codebook depth, can be recovered from those outputs and built into the encoder. The first fact is

a matter of measurement: a fixed stride is wrong by a frame within a few chunks, and the exact rule is confirmed by the generator’s own tables and by an independent implementation. The second is a matter of architecture: the same encoder features support seven independent classifiers or one causal decoder, and only the latter produces code tuples that condition the generator closely. Condition-replay cosine is the measurement that revealed this; it is cheap relative to rendering audio and stops exactly where the generator’s own contribution begins. What we have shown is a functional surrogate of the withheld encoder on the generator’s own distribution, not a recovery of the original component, and the surrogate’s behaviour on recorded music remains to be measured.

Two results argue against natural extensions. The 3.8× larger v2 configuration moved replay cosine by 0.007, and aligning to a strong music representation moved it by 0.0004, so neither capacity nor external audio features was the limit within this family. The remaining gap to the 0.9999 control is dominated by the exposure gap of the depth decoder, and closing it is a decoding problem before it is a representation problem.

10. Limitations and Ethical Considerations

Domain. The corpus is synthetic: every training and evaluation track was generated by MiniMax Music 3 from a narrow set of genre captions, and generalisation to recorded music is not established. Anecdotally the encoder is usable on real references, but we report no metric for it.

Open validation. Three measurements would bound the evidence and were not available to us, because they require the official language model, depth decoder and condition encoder as a scoring path. First, a floor for condition-replay cosine: random codes, codes from another track, and tuples with only the semantic or only the acoustic codes correct would calibrate what 0.87 means against 0.66 for the community encoder and 0.9999 for the true codes. Second, a study on recorded music, scoring functional preservation of a reference (embedding, chroma, tempo or listening judgements) after generation, since no ground-truth codes exist for real audio. Third, an end-to-end check that higher replay cosine corresponds to better preservation after the diffusion and decoding stages. Two matched ablations would sharpen the architectural claim: a 512-wide model trained with the v2 schedule, and a parameter-matched independent-head or non-causal decoder against v4.

Context and decoding. The context is 128 frames (5.12 s) with no state across windows. The depth decoder has a large exposure gap between teacher-forced and greedy decoding, and we tried no beam search, scheduled sampling [3] or differentiable depth sampling. Each architecture was trained once.

Metrics and teacher. Condition-replay cosine stops before the diffusion transformer and is not an audio-quality mea-

sure; exact-token agreement does not identify functionally equivalent code tuples; and the teacher distributions come from the generator’s text-conditioned prior, not from an audio-conditioned posterior.

Ethical considerations. No original encoder weights or source were used; the encoder is trained only on outputs of the released model, obtained through its public interface, and it does not reproduce any withheld weights. It enables reference-conditioned generation, which can be used to imitate a recording’s structure; the released integration constrains only the semantic stream and leaves timbre to the generator, and use of the weights and of any generated audio remains subject to the MiniMax Music 3 terms. The corpus contains no recorded music. The v3 run’s provenance additionally falls under the MERT license.

11. Conclusion

Using the recovered timeline contract and an encoder that decodes causally across codebook depth, we obtain 0.8748 condition-replay cosine on held-out model-generated tracks, compared with 0.7698 for the structurally matched independent-head encoder and 0.6633 for the evaluated community encoder. The results further show that condition replay separates models that exact-token metrics fail to separate, with the depth-causal decoder losing exact acoustic agreement while gaining 0.105 in replay. Calibrating the metric against random codes, measuring preservation on recorded music through the full generation pipeline, and closing the exposure gap of the depth decoder are the next steps.

References

- [1] Andrea Agostinelli, Timo I. Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, Matt Sharifi, Neil Zeghidour, and Christian Frank. MusicLM: Generating music from text. *arXiv preprint arXiv:2301.11325*, 2023. 2
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 4
- [3] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015. 2, 8, and 9
- [4] Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 2
- [5] Bghira. MiniMax Music 3 RVQ reverse-distillation traces. <https://huggingface.co/datasets/bghira/minimax-music3-rvq-reverse-distillation>, 2026. 5

- [6] Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, and Neil Zeghidour. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023. 1 and 2
- [7] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A. Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, Itay Yona, Eric Wallace, David Rolnick, and Florian Tramèr. Stealing part of a production language model. In *International Conference on Machine Learning*, 2024. 1 and 2
- [8] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems*, 2023. 1 and 2
- [9] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024. 4
- [10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022. 4
- [11] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023. 2
- [12] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024. 2
- [13] Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music. *arXiv preprint arXiv:2005.00341*, 2020. 2
- [14] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024. 2
- [15] Zach Evans, Julian D. Parker, CJ Carr, Zack Zukowski, Josiah Taylor, and Jordi Pons. Long-form music generation with latent diffusion. In *International Society for Music Information Retrieval Conference*, 2024. 2
- [16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. 4
- [17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 2 and 4
- [18] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. 2
- [19] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021. 2
- [20] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Conference on Empirical Methods in Natural Language Processing*, 2016. 2
- [21] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. 4
- [22] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems*, 2023. 1 and 2
- [23] Yizhi Li, Ruibin Yuan, Ge Zhang, Yinghao Ma, Xingran Chen, Hanzhi Yin, Chenghua Xiao, Chenghao Lin, Anton Ragni, Emmanouil Benetos, Norbert Gyenge, Roger Dannenberg, Ruibo Liu, Wenhui Chen, Gus Xia, Yemin Shi, Wenhao Huang, Zili Wang, Yike Guo, and Jie Fu. MERT: Acoustic music understanding model with large-scale self-supervised training. In *International Conference on Learning Representations*, 2024. 6
- [24] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 1 and 2
- [25] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. 6
- [26] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. 4
- [27] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. 6
- [28] MiniMax. MiniMax-Music3. <https://huggingface.co/MiniMaxAI/MiniMax-Music3>, 2026. 1 and 2
- [29] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2019. 2
- [30] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *International Conference on Computer Vision*, 2023. 1 and 2
- [31] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 1 and 2
- [32] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021. 4

- [33] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. In *International Conference on Learning Representations*, 2016. 2 and 8
- [34] Serveurperso. minimaxmusic.cpp: an independent 41m RVQ encoder and replay evaluation. <https://github.com/ServeurpersoCom/minimaxmusic.cpp>, 2026. commit d19efe9f. 3, 5, and 13
- [35] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In *USENIX Security Symposium*, 2016. 1 and 2
- [36] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016. 2
- [37] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017. 2
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. 4
- [39] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, Lei He, Sheng Zhao, and Furu Wei. Neural codec language models are zero-shot text to speech synthesizers. In *arXiv preprint arXiv:2301.02111*, 2023. 1
- [40] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision*, 2018. 4
- [41] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020. 4
- [42] Dongchao Yang, Songxiang Liu, Rongjie Huang, Jinchuan Tian, Chao Weng, and Yuexian Zou. HiFi-Codec: Group-residual vector quantization for high fidelity audio codec. *arXiv preprint arXiv:2305.02765*, 2023. 2
- [43] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems*, 2021. 2 and 4
- [44] Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. In *International Conference on Learning Representations*, 2016. 4
- [45] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021. 2
- [46] Xin Zhang, Dong Zhang, Shimin Li, Yaqian Zhou, and Xipeng Qiu. SpeechTokenizer: Unified speech tokenizer for speech language models. In *International Conference on Learning Representations*, 2024. 2
- [47] Liu Ziyin, Tilman Hartwig, and Masahito Ueda. Neural networks fail to learn periodic functions and how to fix it. In *Advances in Neural Information Processing Systems*, 2020. 2

Reverse Engineering the RVQ Audio Encoder of MiniMax Music 3 via Reverse Distillation

Supplementary Material

Table 8. Parameter breakdown of the released encoders.

Component	v1 ($d=512$)	v2, v3 ($d=1,088$)	v4
stem	459,264	975,936	975,936
residual blocks	3,151,872	14,217,984	14,217,984
positions	65,536	139,264	139,264
transformer	25,219,072	113,752,576	113,752,576
output norm	1,024	2,176	2,176
readouts	12,082,176	25,648,128	17,842,176
depth decoder	0	0	22,078,464
total	40,978,944	154,736,064	169,008,576

12. Architecture Details

Per temporal layer $12d^2 + 13d$ parameters (four projections, two LayerNorms, feed-forward $4d$); stem $897d$; each residual block $4d^2 + 4d$; positions $128d$; output norm $2d$; a readout of size V has $dV + V$. The v4 depth decoder: context projection $1,088 \times 512$, prior embeddings $16,384 \times 512 + 6 \times 1,024 \times 512$, depth positions 8×512 , two layers of $12 \cdot 512^2 + 13 \cdot 512$, a norm, and seven heads of $512 \times 1,024 + 1,024$. Table 8 evaluates these identities for the three widths; the totals are the parameter counts of the released weights, and the eight transformer layers hold 62% of v1 and 67% of v4.

13. Additional Experiments

Benchmarks. Figure 5 shows the two measurements that decided the kernel choices of Sec. 5, taken on an NVIDIA GeForce RTX 3090 (sm_86, 23.6 GiB), torch 2.14 on CUDA 13, bfloat16 autocast, tf32 enabled, with latency as the median device time over repetitions. Top: at the temporal stack’s shape the fused kernel is $3.6\times$ faster forward and $2.8\times$ faster backward, and at the depth decoder’s shape it is $2.9\times$ and $3.8\times$ slower, because a sequence of length 8 is below the tile size of the fused kernels and their set-up dominates. Bottom: assembling the pooling operator for a batch on the host costs 152.9 ms as dense matrices and 8.5 ms as segment indices at batch 64, and 19.8 ms against 1.8 ms at batch 16.

Held-out metrics over training. Figure 6 scores all 36 checkpoints of each run on the full held-out split. Semantic top-1 (first panel) climbs steeply for 4,000 steps and then slowly; the three wide models track one another and the 41M model runs two points below them throughout. Acoustic top-1 (second panel) shows the same ordering for the free-running curves, while the dashed teacher-forced v4 curve keeps rising to 18% and opens the exposure gap steadily from the first checkpoint on. The held-out loss of the independent-head models (third panel) is directly comparable across v1 to v3;

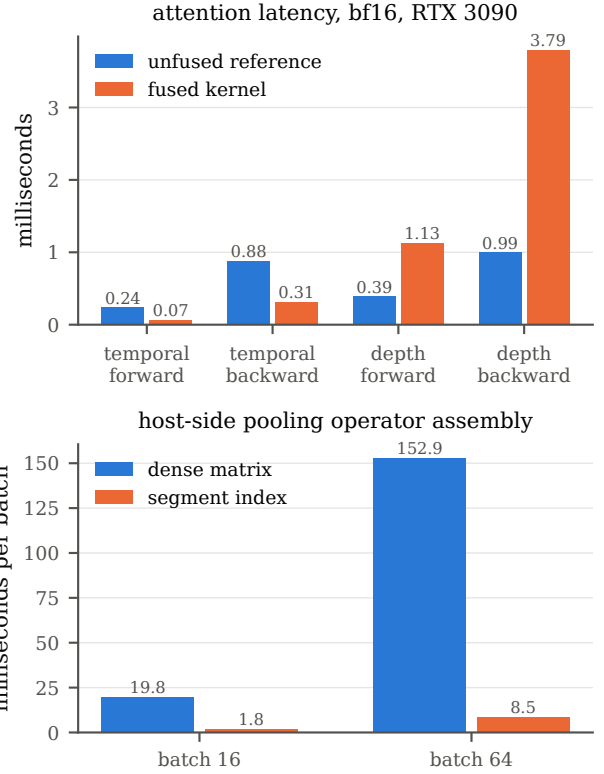


Figure 5. Top: attention latency at the two shapes the model runs; the fused kernel wins on the temporal stack and loses on the depth decoder. Bottom: host-side assembly of the pooling operator per batch, dense matrix against segment index.

the v4 loss (fourth panel) is conditional on the true earlier codes and is drawn on its own axis. All curves are still improving at 17,660 steps.

Training dynamics. Figure 7 shows the training loss (top) as a running mean over 15 logged steps and the learning rate of the width-scaled parameter group relative to its peak (bottom). The loss curves order as the held-out curves do, with v4 lower because its objective is conditional. The learning-rate panel makes the schedule difference between the runs explicit: v1 follows a cosine of half period 500 that returns to its peak every 1,000 steps, whereas v2 to v4 warm up for 500 steps and decay linearly to 10^{-7} ; the v1 loss shows no visible ripple at the reheating points.

Packaged checkpoints. v2 to v4 ship their final checkpoint at step 17,660; their held-out metrics are the published rows of Tab. 9. v1 ships step 17,500 (loss 5.3379, semantic top-1

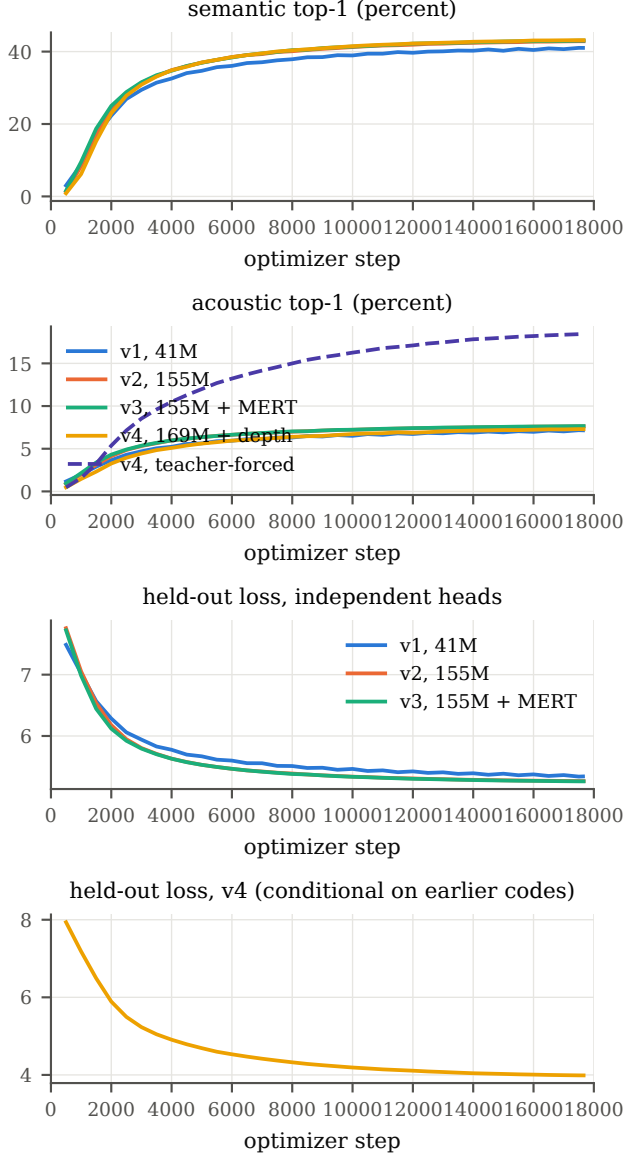


Figure 6. Held-out metrics over the checkpoints of every run: semantic top-1, acoustic top-1, held-out loss of the independent-head models, and the v4 loss, which is conditional on the true earlier codes and has its own panel.

41.03%, acoustic top-1 7.17%), the best of its run by loss and top-1; its final checkpoint at step 17,640 is 0.003 worse in loss, a consequence of the cosine schedule ending mid-cycle. The v1 replay cosine in Tab. 3 was measured on the final checkpoint and the packaged one differs from it by 140 optimizer steps.

Reproducibility. Table 9 places each published held-out row beside the row obtained by re-evaluating the released weights on one RTX 3090, from latents recomputed from the held-out audio. The loader yields the same 135 held-out records, 130 exact, and 2,768 windows. Every published row

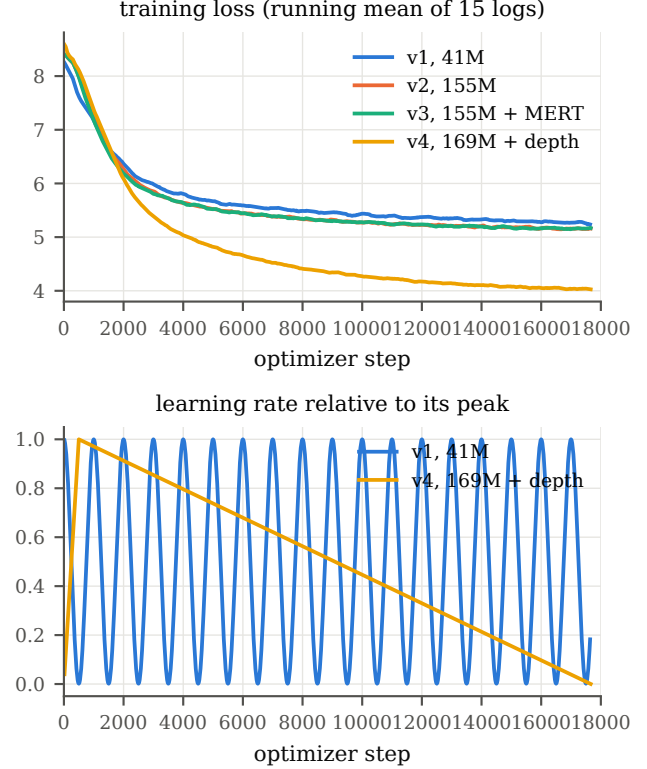


Figure 7. Training loss (top) and learning rate of the width-scaled parameter group relative to its peak (bottom): the v1 cosine schedule reheats every 1,000 steps, v2 to v4 warm up for 500 steps and decay linearly.

is reproduced within 0.003 in loss and 0.001 in accuracy under bfloat16 autocast, the teacher-forced column included, and the full float32 row for v4 moves by a further 0.001; the residual differences are float reassociation across kernels and ranks.

Provenance. Table 10 records where each released weight file came from: the training run (a repository under SimpleTuner/open-rvq-encoder-minimax-music3) and its revision, the checkpoint that was packaged, and the leading digits of the weight file’s checksum, together with the dataset revision used for every number in this paper. Replay evaluation used the official components at revision fbdf52fb, and the community checkpoint is best.pt from commit d19efe9f of minimaxmusic.cpp [34].

Table 9. Published rows against our implementation on the same held-out split.

Model	Source	Loss	Sem. top-1	Sem. top-5	Ac. top-1	Ac. top-5	TF ac. top-1
v1, 41M	published, bf16, 4 ranks	5.3379	41.03	7.17			
v1, 41M	this repository, bf16, 1 GPU	5.3353	41.08	7.18			
v2, 155M	published, bf16, 4 ranks	5.2644	42.86	7.62			
v2, 155M	this repository, bf16, 1 GPU	5.2644	42.86	7.62			
v3, 155M + MERT	published, bf16, 4 ranks	5.2599	43.03	7.66			
v3, 155M + MERT	this repository, bf16, 1 GPU	5.2574	43.07	7.67			
v4, 169M + depth	published, bf16, 4 ranks	3.9874	43.17	7.29	18.42		
v4, 169M + depth	this repository, bf16, 1 GPU	3.9860	43.24	7.30	18.44		
v4, 169M + depth	this repository, fp32, 1 GPU	3.9858	43.26	7.30	18.44		

Table 10. Release provenance.

Version	Source run	Revision	Checkpoint	Weights sha256
v1	41m-v1	e9fa3bba9b5b	checkpoint-17500	9683ba0a719fc8f4
v2	155m-v2	f317d72466b9	final	47dffb7a76d9558
v3	155m-v3	7c525705817d	final	356e97fca65c486a
v4	169m-v4	b9a9165b99d1	final	e8fa93a7db2e5a09
dataset	minimax-music3-rvq-reverse-distillation	5029b1c7f1bb	holdout	

14. Hyperparameters

Table 11 lists every setting of the four runs; the values are those of the released configurations, and the rows marked with a version apply to that run only.

Table 11. Training and evaluation hyperparameters shared by all runs unless noted.

Setting	Value
hardware	4 x NVIDIA L40S, PyTorch DDP
precision	bfloat16 autocast, tf32 matmul
epochs, optimizer steps	20, 17,660 (v1: 17,640)
batch per rank, global	16, 64
optimizer	AdamW under μP , $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$
learning rate, weight decay	3×10^{-4} , 0.01
schedule, v1	cosine, half period 500 steps, no warm-up
schedule, v2 to v4	linear warm-up 500 steps, linear decay to 10^{-7}
gradient norm limit	1.0
μP base width, multiplier	128; 4 (v1), 8.5 (v2 to v4)
attention scale	$8/d_h$, $d_h = 64$
dropout	0.1
window, stride	128 frames, random crop (train), 128 (validation)
KL weight, temperature, top- k	0.25, 1.0, 50
MERT term, v3 only	layer 4 to teacher layer 9, weight 0.5 to 0.7 T , zero by 0.9 T
depth decoder, v4 only	width 512, 2 layers, 8 heads, feed-forward 2,048
validation, checkpoint interval	500 steps
seed	42

15. Audio Examples

Reference-conditioned generations produced with the released v4 encoder through the official integration are published with the weights: a 30 s reference constrained at every fifth semantic frame, and a full-length reference constrained at every fifth and at every tenth frame, each rendered as 44.1 kHz stereo. The examples use the top-5 semantic candidates of the encoder; the acoustic codebooks are generated by Minimax Music 3 and are not copied from the reference.